

1. Duplicate Keys **06A2**

如果允许BST中节点（的关键码）**相等**，你现在能想到什么解决方案？对应的时间、空间效率如何？

2. Besides Inorder **06A2**

BST可以看作是以关键码为序组织起来的一个数据集合，这种次序与BST的**中序**遍历序列相对应。

如果改为与**先序**、**后序**或**层次**遍历序列相对应，会分别遇到什么问题？

3. BST::search() **06B1**

我们记得在Vector::search()二分查找算法里，每一步迭代中所做比较的**次序**、**次数**都颇为讲究，并做过仔细测算与调校。然而，为何在此处实现的BST::search()，却没有过多地考虑这些？

4. Fingers in BST **06B1**

BST::search()算法在内部记录_p、_x和_r这三个节点的位置（犹如三根**手指**）后，可使后续的insert()、remove()等操作简捷而高效地实现。如果不维护此类信息，还有什么**其他**技巧也可达到这一目的？

5. Inserting The First Node **06B2**

BST::insert()算法会首先调用search()，希望找到_x==nullptr及其父节点_p，然后在此局部完成新节点的创建及接入。那么特别地，这种机制是如何保证**首个**节点也得以顺利插入的呢？

- 编译运行示例代码中的BST工程，**确认**这类情况可以得到正确处置；
- 阅读并跟踪代码，**了解**其原理。

6. Time Cost Of BST::remove() **06B3**

我们知道，调用insert()将一个节点引入BST，所需的时间取决于该节点的**深度**。那么，调用remove()从BST中摘除一个节点所需的运行时间，除了节点的深度还与**哪些**因素有关？

7. updateHeightAbove() **06B3**

remove()算法在摘除目标节点之后，会调用_p->updateHeightAbove()来更新祖先节点的高度。然而不难注意到，在双分支的情况下，**实际**被摘除的节点s会比_x**更深**。

- 在这种情况下，是否会有一些祖先节点**不能**及时更新高度？
- 试阅读示例代码，**确认**你的判断。

8. Concurrent remove() **06B3**

我们知道，remove()算法为处理双分支的情况，不得不在一段时间窗口内，在局部**违反**BST的中序顺序性。然而在**并发**环境中，同一棵BST需要支持**多个**应用的**同时**访问。不难理解，尽管这类时间窗口通常很短，但仍然非常有可能造成其它应用的操作出错，即便限定其他应用此时只能做search()，仍有这类风险。

- 为**避免**这类问题，你觉得可以采取什么措施？
- 按照你的方法，在并发环境中BST各操作接口的性能，还将受到哪些**因素**的影响？
- 针对这些因素，你的方法还需做哪些**改进**？

9. Insertion Sequences **06C1**

本节介绍了此前有人为估计出BST的期望高度，将一组关键码的每一排列作为一个插入序列，并分别统计由此生成的BST高度。然而即便从讲义中的简单实例我们也可看出，**同一**棵BST可能对应于**多个**插入序列。

- a) 试说明，**越是平衡**的BST，所对应的插入序列也**越多**；
- b) 试通过编程并实验统计，**验证**上述结论。

10. Number Of BST's**06C1**

试证明：由 n 个互异节点组成的BST，共有 $\text{catalan}(n)$ 棵。

11. Random Sequence**06C1**

讲义中指出，理想随机的插入序列很难出现，实际上其中往往含有明显的**非随机性**。试分别验证，讲义中罗列的各种非随机性（单调性、局部性、周期性，等等），都会倾向于导致BST的**不平衡**。

12. $O(1)$ -Time Rotate**06C3**

试验证：无论在BST中的任何位置做一次zig/zag旋转，都可以在**常数**时间内完成。

13. Transform By Rotates**06C3**

- a) 试证明：借助适当的zig/zag旋转，任何一棵都可以**转换**为另一棵；
- b) 在最坏情况下，这样的转换需做**多少次**旋转？
- c) 试设计一个算法，通过**尽量少**的旋转，完成上述转换；
- d) 编程实现你的算法，并**验证**你的结论。

14. Asymptotically Balanced**06D1**

本节证明了，AVL树的高度不会超过 $O(\log n)$ ，也就是所谓的**渐近平衡**。

- a) 如果将AVL的准则再退让**一步**，即改为： $\forall v \in \text{AVL}, |\text{balFac}(v)| \leq 2$ ，是否还是渐近平衡的？
- b) 如果退让**常数步**呢？试证明你的结论。

15. Size vs. Height**06D1**

我们知道，AVL树中任何一对兄弟子树，**高度**都彼此接近——然而，这并不意味着二者的**规模**也相近。

- a) 在高度为 h 的AVL树中，左、右子树的规模之差**最大**可能达到多少？
- b) 编程、手绘或借助演示工具，为你的答案提供佐证。

16. Size/Weight Balanced**06D1**

我们已经看到，通过限制所有节点的平衡因子有界，AVL成功地保证了整体的渐近平衡。但稍加推敲便不难发现，其对平衡因子的定义未免显得**迂回**。实际上，人们最早尝试的是一种更自然更**直觉**的尺度，即用左、右子树的**规模**（而非高度）之差作为平衡/失衡程度的度量：

$$\text{balFac}(v) = \text{size}(\text{lc}(v)) - \text{size}(\text{rc}(v))$$

这也称作**规模平衡**（size-balanced）或**权重平衡**（weight-balanced）。

有趣的是，这方面的探索后来都逐渐**湮没**于学术历史的长河中，你认为这可能是什么原因？

17. remove() In Fibonaccian Trees**06D1**

本节考查过**最瘦**的AVL树，这类树中所有内部节点的平衡因子都是+1，且它们的规模（大致）按Fibonacci数列而增长，所以也称作**Fibonacci树**。

- a) 实际上，在高度为 h 的Fibonacci树中删除一个节点，有可能会（相继地）引发 $h/2$ 个节点失衡，相应地也需做 $h/2$ 次旋转调整。试指认，**哪个**节点的删除会导致这一极端情况？

- b) 对任何 h ，试构造一棵高度为 $2h$ 的AVL，从中删除某个特定节点之后，可能需要旋转 $2h$ 次；
- c) 对任何的 $n = \text{fib}(h + 3) - 1$ ，试确定 $\{1, 2, 3, \dots, n\}$ 的一个**排列**，使得按照该排列的次序逐个地将 n 个元素插入至一棵初始为空的AVL树，便可以生成一棵高度为 h 、规模为 n 的Fibonacci树。

18. AVL::insert()**06D2**

考查在AVL中（按常规BST的算法）刚刚**接入**一个节点，尚未重平衡的时刻。

- a) 为何只有其祖先**才可能**失衡？
- b) 为何其父亲（ $_p$ ）**不会**失衡？
- c) 何时除父亲之外的**所有**祖先可能同时失衡？
- d) 如果祖先 a_1 与 a_2 失衡，它们之间的其他祖先是否**必然**失衡？

19. AVL::remove()**06D2**

考查在AVL中（按常规BST的算法）刚刚**摘除**一个节点，尚未重平衡的时刻。

- a) 为何只有祖先**才可能**失衡？
- b) 为何**至多**只有一个祖先失衡？
- c) 何时父亲（ $_p$ ）**就可能**失衡？

20. AVL::insert()**06D3**

考查在AVL中（按常规BST的算法）刚刚**接入**一个节点，尚未重平衡的时刻。

- a) g 和 p 的平衡因子可能有哪些**组合**？
- b) 二者会否因平衡因子为 0 ，而导致 $\text{tallerChild}()$ 的**歧义**？
- c) 如果出现歧义，你认为应该如何**破解**？示例代码是这样处理的吗？

21. AVL::insert()**06D3**

在按常规BST算法**接入**一个节点之后，该算法会**自下而上**检查所有祖先，并通过适当的**旋转**使失衡者复衡。

- a) 试确认：如此旋转之后，局部子树的高度**必然**复原；
- b) 试确认：一旦刚做过旋转，便可以**随即**终止上溯；
- c) 试确认：无论是否做过旋转，只要局部子树的**高度未变**，便可随即终止上溯。

我们注意到，示例代码以“ $0 == \text{BalFac}(g)$ ”作为上溯终止的条件。

- d) 试验证：**一旦**做过一次旋转调整，如此的确可以**随即**终止上溯；
- e) 试验证：即便**尚未**做过旋转调整，如此也可以在**适当**的时候终止上溯。

22. AVL::remove()**06D4**

考查在AVL中（按常规BST的算法）刚刚**摘除**一个节点，尚未重平衡的时刻。

- a) g 和 p 的平衡因子可能有哪些**组合**？
- b) 二者会否因平衡因子为 0 ，而导致 $\text{tallerChild}()$ 的**歧义**？
- c) 如果出现歧义，你认为应该如何**破解**？示例代码是这样处理的吗？

23. AVL::remove()**06D4**

在按常规BST算法**摘除**一个节点之后，该算法会**自下而上**检查所有祖先，并通过适当的**旋转**使失衡者复衡。

- a) 试确认：如此旋转之后，局部子树的高度**未必**复原；
- b) 试确认：即便刚做过旋转，也**不能**随即终止上溯；
- c) 试确认：无论是否做过旋转，只要局部子树的**高度未变**，便可随即终止上述（算法正是如此实现的）；
我们注意到，示例代码以 “ $\theta \neq \text{BalFac}(g)$ ” 作为上溯终止的条件。
- d) 试验证：任何一次旋转调整之后，只要局部子树的**高度复原**，如此便可**随即**终止上溯；
- e) 试验证：即便**尚未**做过旋转调整，如此也可以在**适当**的时候终止上溯。

24. AVL::connect342()**06D5**

对于上述tallerChild()的**歧义**情况，connect342()的是如何**破解**的？